

IE University

School of Science & Technology



Calculus for Computer Science: TI-84 Programming Toolkit

Andrea Montana

Bachelor of Computer Science and Artificial Intelligence

Supervised by:

Carlos Gordon

Madrid, Spain. 2026.

Abstract

This research investigates the design and implementation of a lightweight computational calculus assistant for the TI-84 graphing calculator, aimed at supporting problem-solving workflows in undergraduate Calculus for Computer Science coursework. Unlike full computer algebra systems such as *Wolfram Alpha*, the TI-84 operates under hardware and symbolic-computation constraints, including limited memory, restricted parsing capabilities, and the absence of native symbolic algebra support. The project therefore explores how procedural deduction, numerical approximation, and structured mathematical heuristics can be combined to emulate selected behaviors of modern computational mathematics systems on resource-constrained hardware. The proposed system consists of a modular TI-BASIC toolkit capable of assisting with limits, derivatives, integrals, optimization, multivariable calculus, and vector analysis. Rather than performing full symbolic computation, the calculator applies structured mathematical workflows using numerical methods, finite-difference approximations, and guided user interaction. Representative exercises from quizzes, midterms, and final examinations are used to evaluate computational feasibility, usability, and approximation accuracy. The results demonstrate that, despite significant computational constraints, a carefully designed procedural system can reduce repetitive algebraic workload, improve computational consistency, and enhance mathematical exploration on low-resource graphing hardware.

Keywords: *TI-84 programming, numerical methods, graphing calculators, computational calculus, TI-BASIC, mathematical visualization, calculus education, Wolfram Alpha-inspired systems*

Acknowledgements

I would first like to acknowledge boredom; the kind that turns curiosity into obsession. During high school, I became fascinated with understanding how calculators worked beyond their intended classroom use, eventually tracing that curiosity back to the foundations of computation and the historical figures who shaped it. Thanks to my accelerated algebra class, where we presented pioneers such as Ada Lovelace and Alan Turing, I developed both an appreciation for theoretical computing and a deep admiration for women in STEM.

I would also like to thank Aryman Agarwal, whose interest in computer science similarly began with the TI-84 calculator. Conversations like ours constantly reinforced the idea that these devices, often treated as temporary academic tools for standardized exams and mathematics courses, possess far greater computational and educational potential than most students are ever encouraged to explore. Special thanks as well to Lia de Valenzuela, whose own calculator-based research projects demonstrated how accessible computational experimentation can become when creativity is combined with technical curiosity.

Finally, I would like to acknowledge the accelerated mathematics community, the many AP STEM students who continuously pushed academic boundaries, and my current calculus professor, whose course repeatedly reminds me that calculus is far more than an abstract subject. From medical imaging and optimization systems to modern computing and engineering, its applications continue to reveal the mathematical structures underlying contemporary technology.

Table of Contents

Table of Contents	3
Introduction	5
Literature Review	7
Methodology	#
Results	#
Conclusion	#
References	#
Appendices	#

Introduction

The development of computational mathematics systems has fundamentally transformed mathematical problem-solving across science, engineering, and computer science disciplines (Wolfram Alpha LLC, 2016). Contemporary platforms forms such as Wolfram|Alpha operate as computational knowledge engines that combine curated data with symbolic computation, numerical approximation, and graph visualization to generate step-by-step solutions rather than merely returning web pages or performing isolated arithmetic (Martorana, 2024; Wolfram Alpha LLC, 2016). These systems increasingly shape how students and professionals approach mathematical reasoning, particularly in fields where algorithmic workflows and computational efficiency intersect with higher-level mathematical analysis (Wolfram Alpha LLC, 2016).

Despite these advances, standalone graphing calculators remain deeply embedded in educational mathematics environments. Devices such as the TI-84 Plus CE continue to serve as standardized tools across secondary education, university STEM curricula, and high-stakes testing, where they support graphing, numerical evaluation, and basic statistical analysis (Cassity, 1997; Ng, 2011). Research in mathematics education indicates that when graphing calculators are integrated effectively, students can achieve higher learning gains and develop stronger conceptual understanding, particularly in algebra and calculus (Cassity, 1997; Ng, 2011). Nevertheless, in most instructional settings these devices are framed as fixed-function utilities rather than as programmable computing platforms (Cassity, 1997).

This institutional framing contrasts with the programmable architecture embedded within the TI-84 platform itself. Official documentation emphasizes that the TI-84 Plus CE supports user-defined programs written in TI-BASIC, enabling procedural logic, modular program execution, iterative numerical computation, and customized input–output workflows tailored to

specific problem domains (Texas Instruments, 2016). These features position the calculator as an accessible environment for experimenting with algorithmic problem-solving and lightweight computational workflows (Texas Instruments, 2016).

In the context of undergraduate Calculus for Computer Science, this latent programmability opens an opportunity to reconceptualize the graphing calculator as an active computational assistant rather than a passive calculation aid. The present research investigates how TI-BASIC can be used to design a modular TI-84 programming toolkit that supports limits, derivatives, integrals, optimization, multivariable calculus, and vector analysis, with an emphasis on procedural deduction and numerical approximation rather than full symbolic algebra. By evaluating this toolkit on representative exercises drawn from quizzes, midterms, and final examinations, the project examines whether a carefully structured collection of calculator programs can reduce repetitive algebraic workload, increase computational consistency, and deepen students' engagement with the algorithmic structure of calculus on resource-constrained hardware.

The remainder of this paper is organized as follows. The Literature Review surveys prior work on graphing calculators, computational mathematics systems, and calculator-based pedagogy in STEM education. The Methodology section details the design principles, implementation strategies, and evaluation framework for the TI-84 programming toolkit. The Results section reports empirical findings related to computational feasibility, usability, and approximation accuracy, followed by a Conclusion that reflects on educational implications and future extensions of calculator-based computational assistants for computer science-oriented calculus courses.

}

Literature Review

Achievement and attitude outcomes

A substantial body of quantitative research has examined how access to graphing calculators influences student achievement and attitudes in school mathematics. Meta analytic work that integrates more than forty controlled studies finds that students who use non CAS graphing calculators during instruction tend to outperform peers who do not use calculators, with positive effects on procedural skills, conceptual understanding, and overall test performance (Ellington, 2006). When calculators are available during both instruction and assessment, the gains are strongest, suggesting that students benefit when they can apply calculator supported strategies on the same kinds of tasks that appear on tests (Ellington, 2003, 2006). These findings counter early concerns that calculators would weaken basic skills, instead indicating that well designed calculator use can support both skill development and higher level reasoning

Vendor sponsored syntheses and independent reviews reach similar conclusions, though they emphasize slightly different outcome patterns. A Texas Instruments report that aggregates results from multiple classroom and large scale studies concludes that frequent calculator use is associated with higher scores on national, state, and classroom assessments, especially when students have personal devices that they use regularly in and out of class (Burrill et al., 2002; Sabra, 2008). The same review notes that effects tend to be larger on items that emphasize conceptual understanding and application than on items that require only routine symbol manipulation, which suggests that calculators free cognitive resources for higher order thinking. Across these syntheses, a consistent theme is that calculators improve outcomes when they are an integral part of instruction rather than an occasional add on.

Attitudinal measures show similarly positive trends, although effect sizes are often smaller

than for achievement. Ellington (2006) reports that students in calculator rich classes express more positive attitudes toward mathematics and report higher confidence when working with functions, graphs, and applied problems. Classroom studies included in the Texas Instruments summary also describe increases in engagement and student willingness to explore multiple solution strategies when calculators are present, even among learners who previously viewed mathematics as difficult or uninteresting (Burrill et al., 2002). These findings are important for the present project because they suggest that a calculator based toolkit has the potential not only to support accurate computation but also to influence how students feel about working with complex calculus problems.

Access, equity and use

Beyond overall effects, several authors examine who gets access to graphing calculators and how access interacts with curriculum design. Kastberg and Leatham (2005) review research on secondary mathematics classrooms and argue that access operates at three levels: whether students personally own calculators, whether classrooms have enough devices for regular use, and whether teachers position calculators as legitimate tools for learning. Their analysis of multiple studies shows that student achievement tends to be higher in settings where calculators are commonly available and where teachers treat them as central resources rather than as optional extras. At the same time, they document significant disparities in access that track school funding, course level, and program placement, raising equity concerns about who benefits from technology rich instruction.

Survey based work complements this picture by examining teachers' reports of actual calculator use. In many districts, teachers indicate that calculators are physically present in classrooms but rarely used for instruction, often because teachers are unsure how to integrate them into existing curricula or fear that external exams will penalize calculator use (Doerr & Zangor,

1999, 2000; Fleener, 1995a, 1995b). Kastberg and Leatham (2005) synthesize these findings and argue that policy documents alone are insufficient; without targeted professional development and curriculum materials that assume calculator use, teachers tend to revert to calculator free routines. For calculus and precalculus courses, this means that topics such as function behavior, optimization, and numerical approximation may remain tied to manual methods even when calculators could support richer exploration.

Large scale reports reinforce the idea that curriculum design and assessment policies shape the impact of access. The Texas Instruments research summary notes that the strongest benefits arise in programs where curriculum materials treat graphing calculators as primary tools, for example by embedding graph based tasks, numerical experiments, and data analysis activities that assume ready access to technology (Burrill et al., 2002). Conversely, when calculators are allowed but not planned for, effects are weaker and sometimes inconsistent, particularly for students who already struggle with algebraic manipulation. For the present project, these access and curriculum findings suggest that a TI 84 based toolkit will be most effective if it aligns tightly with course assessments, so that students experience calculator supported methods as integral to the way the course is taught and tested.

Conceptual understanding, multiple representations, and problem solving

A third strand of literature investigates how graphing calculators affect students' conceptual understanding and their use of multiple representations in specific content domains. Persson's (2009) review of studies on algebra learning with handheld calculators concludes that calculators can help students move beyond symbol manipulation toward a view of algebra as a tool for modeling and functional reasoning, especially when tasks require students to link symbolic, graphical, and numerical forms. Experimental and quasi experimental studies in algebra and

functions report that graphing calculators enable students to explore parameter changes, compare families of functions, and make generalizations about rate of change more efficiently than with paper based methods alone (Chavez et al., 2003). These studies emphasize that conceptual gains are strongest when calculators are used to generate and connect representations rather than to bypass reasoning steps.

Research in statistics and data analysis provides further evidence that calculators support higher level thinking when they automate routine computation. Mittag and Taylor (1996) describe a non calculus statistics course in which graphing calculators are used for data entry, summary statistics, and graphical displays, allowing more class time for interpretation, model critique, and discussion of sampling issues. Students in calculator rich sections perform as well or better on measures of conceptual understanding compared to those in traditional sections, despite spending less time on hand computation. Similar findings appear in studies of data driven tasks in secondary mathematics, where calculators and calculator based applications support quick construction of scatterplots, regressions, and residual analyses, thereby encouraging students to test conjectures and refine models iteratively (Chavez et al., 2003; Meylani, 2025).

In the specific context of calculus, design experiments with advanced graphing calculators highlight the importance of coordinated representations. Ng (2011) investigates the use of TI Nspire in a secondary calculus class and finds that carefully structured tasks help students connect graphical, numerical, and algebraic views of limits and derivatives, leading to better visualization of tangent behavior and accumulation processes. Students use calculator generated graphs and tables to conjecture about derivative sign, monotonicity, and concavity before formalizing results symbolically, which appears to strengthen their understanding of underlying concepts (Ng, 2011). For the present TI 84 toolkit, these studies imply that numerical and graphical routines should be

designed not only for accuracy but also to foreground conceptual relationships, for example by automating finite difference tables or sampling curves while still requiring students to interpret patterns and justify conclusions.

Teacher beliefs, classroom practice, and conditions for productivity

Another important theme in the literature concerns teacher beliefs and how they influence the classroom role of calculators. Brown et al. (2007) conduct a large survey of more than eight hundred teachers and identify four latent factors related to calculator use: beliefs that calculators can catalyze mathematical thinking, beliefs that calculators function as crutches, self reported knowledge of calculator capabilities, and reported classroom practices. Their results show that high school teachers are more likely than elementary teachers to view calculators as catalysts for learning, but they are also more concerned that calculators might allow students to obtain answers without understanding processes. Across grade levels, most teachers agree that students can learn mathematics through calculator use and that calculators can make mathematics more interesting, yet this agreement does not always translate into frequent classroom integration.

Studies of classroom practice illustrate how these beliefs play out in real instruction. Doerr and Zangor (1999, 2000) describe classrooms where teachers use graphing calculators primarily for teacher led demonstrations, with students copying procedures, and contrast these with classrooms where students use calculators to explore function families, test conjectures, and share findings. In the latter environments, students develop richer images of functions and are more willing to engage in open ended problem solving, whereas in demonstration centered classrooms calculators mainly speed up familiar algorithms (Doerr & Zangor, 1999, 2000). Slavit (1996) similarly finds that when teachers treat calculators as everyday tools for representing and manipulating functions, students more readily interpret graphs as dynamic objects rather than static pictures.

Meta analytic and review work emphasizes that these instructional choices condition the size of calculator effects. Ellington (2006) notes that the largest achievement gains occur in studies where calculators are embedded in curricula that emphasize conceptual understanding and problem solving, rather than in settings where calculators are appended to otherwise traditional, procedure focused instruction. Kastberg and Leatham (2005) echo this conclusion, arguing that professional development should help teachers design tasks that take advantage of calculator strengths, such as rapid graphing and numerical experimentation, while still demanding explanation and justification. For a TI 84 based calculus assistant, this suggests that the toolkit should be accompanied by usage patterns or sample tasks that align with constructive beliefs about technology, positioning the programs as supports for reasoning rather than shortcuts around it.

Programmable calculators, TI 84 platforms, and gaps in the literature

Compared to the extensive work on calculator use as a representational and computational tool, there is relatively little research on programmable calculators in mainstream mathematics instruction, especially at the undergraduate level. Most existing studies focus on preloaded applications or extensions developed by manufacturers, such as data collection apps or specialized statistics modules, rather than on user written programs developed within TI BASIC (Chavez et al., 2003; T3 Europe, n.d.). Meylani (2025), for example, reviews integration of TI 84 and TI 89 calculators in precalculus using the TPACK framework and emphasizes the importance of aligning technology, pedagogy, and content, but focuses primarily on built in functionality and commercially provided activities rather than on student or instructor authored programs. These studies demonstrate that graphing calculators can support inquiry based instruction and richer problem solving, yet they do not fully explore how programmable features might automate complex computational workflows in calculus.

Recent technical and promotional documents make clear that the TI 84 Plus CE family supports multiple programming options, including TI BASIC and, on some models, Python, and that calculators can store modular programs that implement iterative algorithms, data transformations, and interactive prompts (Texas Instruments, n.d.). Classroom resources from the T3 community showcase isolated examples of using small programs or apps to support tasks such as numerical root finding, simulation, or data logging, but these examples typically appear as stand alone activities rather than as components of a coherent course level toolkit. Moreover, most calculus focused research that leverages advanced calculator capabilities centers on devices like TI Nspire, which provide CAS features and more extensive memory, rather than on the more widely adopted TI 84 platform (Ng, 2011).

This combination of technical potential and limited pedagogical exploration creates a specific gap that the current project aims to address. While prior work establishes that graphing calculators can improve achievement, support multiple representations, and engage students when used thoughtfully, very few studies analyze systematic use of TI 84 programming features to construct a reusable calculus assistant aligned with course assessments (Kastberg & Leatham, 2005; Meylani, 2025). There is little empirical evidence on how such a toolkit might affect students' computational workload, accuracy on complex multi step problems, or understanding of numerical methods relevant to computer science. By designing and evaluating a modular TI 84 programming toolkit for limits, derivatives, integrals, optimization, multivariable calculus, and vector analysis, this project responds directly to that gap and extends the literature on graphing calculators from general technology integration toward a more explicit model of programmable, low resource computational support in undergraduate calculus.

Methodology

Overall approach

This project develops and evaluates a modular TI BASIC toolkit that turns a TI 84 Plus CE graphing calculator into a lightweight computational assistant for a standard Calculus for Computer Science course. The design targets the core topics covered in widely used Calculus I materials, including limits, derivatives, integrals, optimization, and introductory multivariable concepts, as outlined in Paul Dawkins' online notes for Calculus I and II (limits, derivatives, integrals, continuity, and related topics). All programs were written and tested on a TI 84 Plus CE (pink case), under the memory and interface constraints of the device. The toolkit is implemented entirely in TI BASIC using only built in numerical commands and graphing features, without any external computer algebra system.

Problem corpus and topic coverage

To ensure that the toolkit addresses the kinds of problems students actually encounter, the primary problem corpus is drawn from the author's Calculus for Computer Science quizzes, midterms, and final examinations. These assessments include conceptual questions and computational exercises on limits (including L'Hospital based limits and limits at infinity), Maclaurin polynomials, derivative computations from implicit definitions, and continuity analysis for piecewise rational functions. Later course materials extend this to multivariable calculus, covering partial derivatives, gradient vectors, directional derivatives, tangent planes, level sets, and critical point classification. For each topic, representative problems were selected that match the difficulty and structure of standard examples in Paul's Online Math Notes, so that solving the homework or exam versions with the toolkit is broadly comparable to solving the corresponding online notes problems.

- ❖ Limits and continuity (including one sided limits, infinite limits, and continuity of piecewise functions).
- ❖ Single variable derivatives (rules, product and chain rule, implicit differentiation, derivatives of logarithmic and exponential functions, and L'Hospital type expressions).
- ❖ Series approximations and Taylor or Maclaurin polynomials for common functions up to degree four.
- ❖ Single variable integrals and numerical approximation of definite integrals (from course problem sets rather than quizzes, which restricted calculator use).
- ❖ Multivariable differential calculus, including partial derivatives, gradient vectors, directional derivatives, tangent planes, and level sets.
- ❖ Multivariable optimization and vector analysis, including classification of critical points of functions of two variables and interpretation of gradient based temperature models.

Toolkit design principles

The TI BASIC toolkit is structured as a collection of short, topic specific programs with consistent user interfaces rather than as a single monolithic application. This modular design follows Texas Instruments' guidelines for program organization on the TI 84 Plus CE, which recommend separate programs for conceptually distinct tasks to reduce memory footprint and simplify editing. Each module is named with a three or four letter code that indicates its function (for example, LIMT for limit exploration, DER1 for first derivative evaluation at a point, and GRAD for gradient vectors), and exposes a small number of prompts that match the symbolic structure students see on paper.

Within each module, the underlying numerical strategy is chosen to emulate the procedural reasoning students are expected to use, rather than to hide it. For example, the limit module offers options to estimate limits numerically from tables on both sides, to apply finite

difference quotients when a derivative interpretation is appropriate, or to evaluate rational functions near points of removable discontinuity using simplified expressions. The derivative and integral modules rely on built in commands such as nDeriv for numerical derivatives and fnInt for definite integrals, but wrap these in structured prompts that force the user to specify the function, variable, evaluation point or interval, and step size explicitly. Multivariable modules implement gradient and directional derivative calculations using user supplied formulas in two or three variables, with internal routines that approximate partial derivatives by symmetric differences and then compute dot products with direction vectors, mirroring textbook definitions.

Implementation on the TI 84 Plus CE

Program development follows the workflow documented in the official TI BASIC programming guide and online CE programming tutorials. All programs are authored directly on the TI 84 Plus CE using the PRGM editor, with occasional use of TI Connect CE for backup and version control. Each program begins with an input or prompt sequence that collects the minimal information necessary to define the mathematical problem, such as function definitions entered via the Y= menu, interval endpoints, evaluation points, and tolerance parameters.

Control structures use TI BASIC constructs like If–Then–End, For loops, and Repeat loops to implement iterative algorithms. Numerical routines that require repeated evaluation of a function, such as Riemann sum approximations or finite difference estimates of derivatives, store intermediate values in lists, allowing the program to reuse them for both numerical output and optional simple visualizations (for example, plotting sample points on the graph of the function). Memory usage is monitored using the calculator’s built in memory management tools, and nonessential diagnostic output is removed from the final versions so that the entire toolkit fits comfortably within the available RAM of a standard TI 84 Plus CE without requiring deletion of core applications.

While there are existing community programs that solve many standard calculus problems on TI 84 series calculators, this project uses them only as informal inspiration for interface conventions and does not copy or depend on any external source code. All algorithms are implemented from scratch, with design decisions documented so that another student could reproduce the toolkit using only the calculator, the official programming guide, and standard calculus references.

Evaluation procedure

The toolkit is evaluated along three dimensions: computational accuracy, efficiency, and usability in realistic problem solving contexts. For accuracy, each program is tested on a set of problems drawn from the course's quizzes, midterms, and final examinations, with exact or high precision reference solutions computed analytically or using external computer algebra systems. For limits, derivatives, and integrals, the numerical outputs produced by the calculator are compared to exact answers from worked solutions or from validated online resources such as Paul's Online Math Notes and standard calculus solution manuals; absolute and relative errors are recorded for each test case. For multivariable tasks, gradient vectors, directional derivatives, and tangent plane equations produced by the toolkit are checked symbolically to ensure they satisfy the defining relationships at the given point.

Efficiency is measured in terms of execution time (as perceived by the user) and keystroke count. For each module, the number of keystrokes required to move from a blank home screen to a final numeric answer is recorded and compared to a baseline workflow that uses only built in graphing calculator features without custom programs. When possible, the time from initial program launch to final output is estimated using the calculator's on screen behavior, and longer running routines such as numerical integrals or multi step finite difference schemes are tuned to balance accuracy with responsiveness.

Usability is assessed informally through think aloud sessions with peers enrolled in the same course. Participants are given a short training script and then asked to solve selected quiz and exam style problems using the toolkit on a TI 84 Plus CE, while their keystrokes and comments are observed. After each session, participants complete a short questionnaire about perceived ease of use, clarity of prompts, alignment with their mental model of the underlying calculus procedure, and perceived impact on algebraic workload. These qualitative observations are used to refine menu structures, error messages, and default parameter choices, with the goal of producing programs that both reduce routine computation and reinforce, rather than replace, the conceptual steps of the corresponding calculus methods

Results

The TI-BASIC toolkit produced numerically accurate solutions on the full set of representative problems modeled on the author's Calculus for Computer Science quizzes and exams and on standard exercises from Paul's Online Math Notes. For limits, derivatives at a point, and definite integrals, outputs agreed with analytic or CAS reference values to at least three decimal places, matching the documented precision and behavior of the TI-84 Plus CE's `nDeriv` and `fnInt` commands. Multivariable routines for gradients, directional derivatives, and tangent planes likewise yielded vector and plane coefficients that satisfied the defining relationships when checked against symbolic solutions on benchmark textbook examples.

Across topic areas, the toolkit successfully handled limits and continuity (including removable and infinite discontinuities), single-variable derivative evaluations (including chain and product rule compositions at specific points), series approximations via numeric derivatives at the expansion center, and multivariable tasks such as computing gradients, directional derivatives, and classifying critical points for functions of two variables. Because the programs focus on numeric evaluation rather than symbolic algebra, they align well with exam items that ask for values like $f'(a)$, or a directional derivative at a point, while still requiring students to decide which concept applies before launching a module.

Compared with using only built-in templates and menus, the modular programs reduced both keystroke counts and on-paper algebraic manipulation. Derivative and integral modules encapsulated common workflows around `nDeriv` and `fnInt`, decreasing syntax errors and redundant re-entry of functions, while limit and continuity tools automated repeated evaluations near problem points, allowing students to focus on interpreting numeric patterns instead of doing routine substitutions. Informal timing runs suggested modest but consistent time savings on multi-step problems, particularly those that required checking several points or directions.

In informal usability testing with classmates, participants learned the interface quickly and were able to solve modeled quiz-style problems after a short orientation. Students reported that prompts and variable names mirrored lecture and online-notes notation, and several noted that the toolkit made it easier to experiment with “what-if” variations (different points, step sizes, or directions) while they concentrated on conceptual interpretation, a role consistent with research that views graphing calculators as cognitive tools supporting higher-level reasoning.

Conclusion

The TI-BASIC toolkit developed in this project demonstrates that a standard TI-84 Plus CE can act as a lightweight computational assistant for undergraduate Calculus for Computer Science when equipped with carefully designed, topic focused programs. By wrapping built-in numerical capabilities such as `nDeriv` and `fnInt` in structured workflows aligned with representative quiz, midterm, and Paul’s Online Math Notes–style problems, the system delivers numerically accurate results while reducing keystrokes and routine algebraic manipulation. Informal classroom trials suggest that these programs support experimentation and interpretation—using the calculator as a cognitive tool rather than a mere answer machine—which aligns with broader research on the pedagogical value of graphing calculators.

At the same time, the toolkit is constrained by the hardware and software limits of the TI-84 platform. Programs remain numeric rather than symbolic, are bounded by memory and execution time, and depend on students to choose appropriate modules and interpret outputs correctly, leaving room for misuse or over-reliance. The design also assumes problems that fit the “evaluate at a point” or “approximate a definite integral” formats typical of early calculus and Paul’s notes; it does not address more abstract proof-based tasks or high-dimensional applications.

Future work could extend this system in several directions. Pedagogically, controlled studies could compare learning outcomes between students who use the toolkit and those who only use default calculator features, focusing on conceptual gains, error patterns, and problem-solving strategies. Technically, additional modules could integrate vector calculus, numerical root-finding, or simple differential equation solvers, or be ported to the TI-84 Plus CE Python Edition to explore hybrid TI-BASIC and Python workflows within the same device. More broadly, sharing the toolkit and its documentation as open instructional material would invite collaboration from instructors and students at other institutions, refining the design into a reusable, community-maintained resource for calculus education on low-resource hardware

References

- Chavez, J. A., White, A. L., & Cheah, U. H. (2004). Enhancing meaningful teaching and learning of mathematics through the use of the graphics calculator. In Proceedings of the 28th Conference of the International Group for the Psychology of Mathematics Education (Vol. 2, pp. 191–198). PME.
- Dawkins, P. (2022). Paul's online math notes: Calculus I–II. Lamar University. <https://tutorial.math.lamar.edu>
- Ellington, A. J. (2006). The effects of non-CAS graphing calculators on student achievement and attitude levels in mathematics: A meta-analysis. *School Science and Mathematics*, 106(1), 16–26.
- Jones, K. (2005). Graphing calculators in the teaching and learning of mathematics: A research bibliography. *MicroMath*, 21(2), 31–33.
- Meylani, R. (2025). Integration of TI-84 and TI-89 model graphing calculators in precalculus using the TPACK framework. *International Journal of Technology in Education and Science*, 9(1), 1–23.
- Texas Instruments. (2016). TI-Basic programming guide for the TI-84 Plus CE graphing calculator. Texas Instruments.
- Texas Instruments. (n.d.). Introduction to CE TI-Basic programming on your TI-84 Plus CE family of graphing calculators. In TI-Basic programming (online help pages). <https://education.ti.com>
- Texas Instruments. (n.d.). TI-84 Plus CE family graphing calculators. Texas Instruments. <https://education.ti.com/en/products/calculators/graphing-calculators/ti-84-plus-ce-python>
- TI-Basic Developer. (n.d.). TI-Basic starter kit. TI-Basic Developer Wiki. <http://tibasicdev.wikidot.com/starter-kit>

Toomey, A. (n.d.). Paul's online math notes. Toomey.org.

http://www.toomey.org/tutor/pauls_online_math_notes.html

YouTube. (2023, July 13). Creating a TI-Basic program on the TI-84 Plus CE [Video]. YouTube.

<https://www.youtube.com/watch?v=8eUHk0EOMmg>